Machine Intelligence 1 Tutorial

05 Regularization and Validation

Fachgebiet Neuronale Informationsverarbeitung (NI)

WiSe 23/24

Regularization

- L2 regularization: weight decay
- L1 regularization

Validation

- Overfitting vs. Underfitting
- Improving how we estimate $\widehat{E}^G$

Nonlinear basis functions

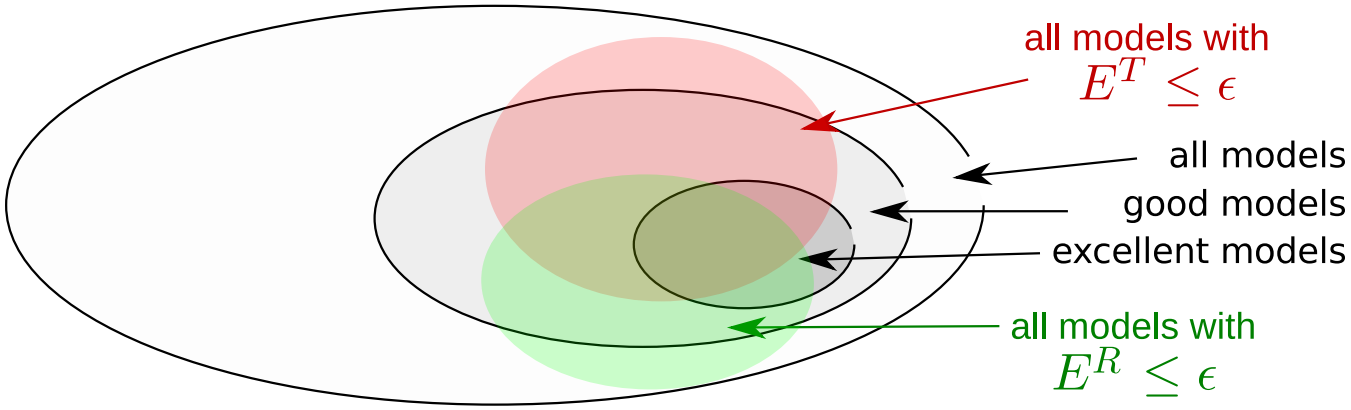
- A non-linearly separable problem
- Nonlinear basis function
- Preprocessing: Sphering
- Linear neuron for regression: Analytical solution

Regularization

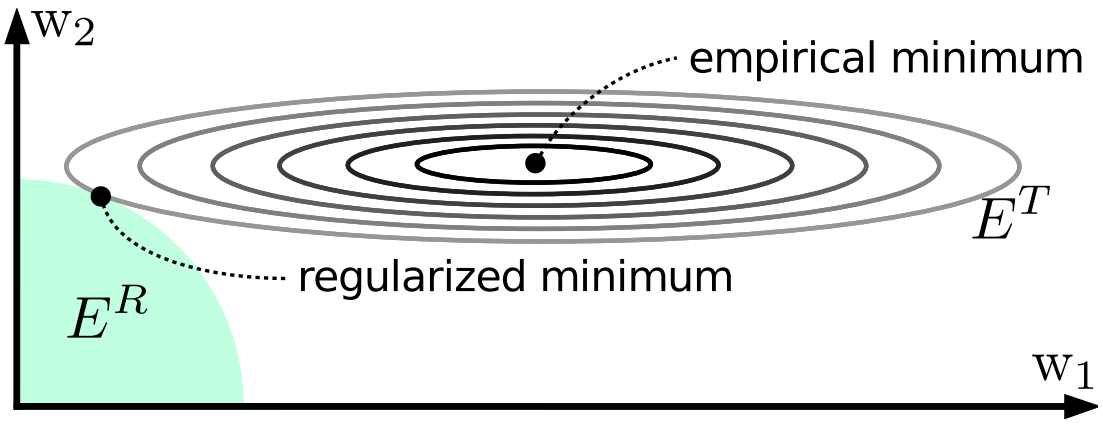
Risk function

$$R_{[\underline{\mathbf{w}}]} = \underbrace{E^T_{[\underline{\mathbf{w}}]}}_{\substack{\text{training}\\ \text{error}\\ \text{e.g. quadratic cost,}\\ \text{cross-entropy,...}}} + \underbrace{\lambda E^R_{[\underline{\mathbf{w}}]}}_{\substack{\text{regularization}\\ \text{term}}} \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$$

- $E^R_{[\underline{\mathbf{w}}]}$: prior knowledge of solution
- λ : regularization parameter



Regularization example: L_2 regularization (weight decay)



$$E^R_{[\underline{\mathbf{w}}]} = \frac{1}{2} \sum_{(i,j,v',v)} \left(w_{ij}^{v'v}\right)^2 = \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 = \frac{1}{2}\underline{\mathbf{w}}^\top \underline{\mathbf{w}}$$

Minimization of $R_{[\underline{\mathbf{w}}]}$ through gradient descent

$$\Delta \mathrm{w}_{ij}^{v'v} \sim -\frac{\partial R_{[\underline{\mathbf{w}}]}}{\partial w_{ij}^{v'v}} = \underbrace{-\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_{ij}^{v'v}}}_{\text{e.g. via backprop}} \underbrace{-\frac{\partial E^R_{[\underline{\mathbf{w}}]}}{\partial w_{ij}^{v'v}}}_{\text{decay term}}$$

L_2 regularization cost

$$E^R_{[\underline{\mathbf{w}}]} := \frac{1}{2} \sum_{(i,j,v',v)} \left(\mathrm{w}^{v'v}_{ij}\right)^2 = \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 = \frac{1}{2}\underline{\mathbf{w}}^\top \underline{\mathbf{w}}$$

Computing individual components of the gradient vector:

$$\begin{aligned}\frac{\partial E^R_{[\underline{\mathbf{w}}]}}{\partial w^{v'v}_{ij}} &= \frac{1}{2} \sum_{(k,l,\mu,\gamma)} \frac{\partial}{\partial w^{v'v}_{ij}} \left(w^{\mu\gamma}_{kl}\right)^2 \\ &= 0 + \ldots + \frac{1}{2}\frac{\partial}{\partial w^{v'v}_{ij}} \left(w^{v'v}_{ij}\right)^2 + \ldots + 0 \;=\; \frac{1}{\cancel{2}}\,\cancel{2}\, w^{v'v}_{ij}\end{aligned}$$

Using vector notation to compute the gradient:

$$\frac{\partial}{\partial \underline{\mathbf{w}}} E^R_{[\underline{\mathbf{w}}]} = \frac{1}{\cancel{2}} \cdot \cancel{2} \cdot \underline{\mathbf{w}} = \underline{\mathbf{w}}$$

Regularized linear neuron for regression: analytical solution

$$R_{[\underline{\mathbf{w}}]} = \underbrace{E^T_{[\underline{\mathbf{w}}]}}_{\substack{\text{training error} \\ \text{e.g. quadratic cost}}} + \underbrace{\lambda E^R_{[\underline{\mathbf{w}}]}}_{\substack{\text{regularization} \\ \text{term}}} \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$$

Gradient:

$$\underline{\mathbf{g}}^R := \frac{\partial}{\partial \underline{\mathbf{w}}} R_{[\underline{\mathbf{w}}]} = \frac{\partial}{\partial \underline{\mathbf{w}}} E^T_{[\underline{\mathbf{w}}]} + \lambda \frac{\partial}{\partial \underline{\mathbf{w}}} E^R_{[\underline{\mathbf{w}}]}$$

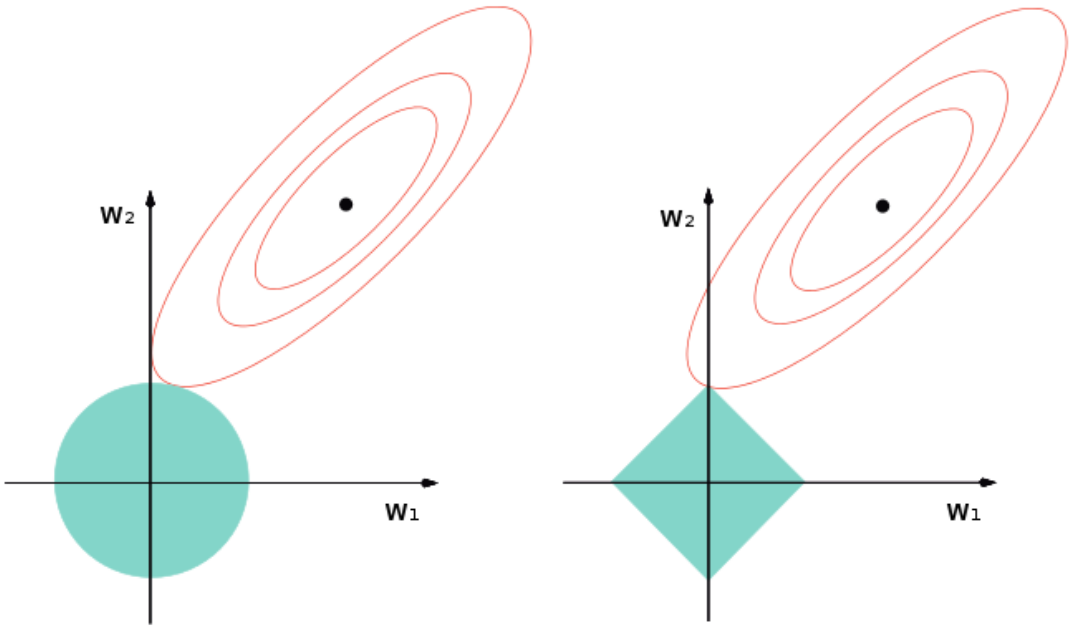
$$= \frac{1}{p} \left(\underline{\mathbf{X}}\, \underline{\mathbf{X}}^\top \underline{\mathbf{w}} - \underline{\mathbf{X}}\, \underline{\mathbf{y}}^\top_{\text{True}} \right) + \underline{\mathbf{I}}_N \, \lambda \, \underline{\mathbf{w}}$$

$$\iff \underline{\mathbf{g}}^R = \underline{\mathbf{0}}$$

$$\underline{\mathbf{w}}^* = \underbrace{\left(\underline{\mathbf{X}}\, \underline{\mathbf{X}}^\top + p\lambda \, \underline{\mathbf{I}}_N \right)}_{\substack{\text{positive definite and} \\ \text{therefore invertible}}}^{-1} \underline{\mathbf{X}}\, \underline{\mathbf{y}}^\top_{\text{True}}$$

L1 regularization

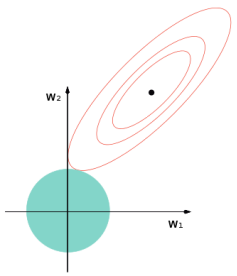
$$E^R_{[\underline{\mathbf{w}}]} = \sum_{(i,j,v',v)} |w_{ij}^{v'v}|^1$$



L_2 *weight decay* L_1 *lasso*

"Lasso" stands for "**l**east **a**bsolute **s**hrinkage and **s**election **o**perator".

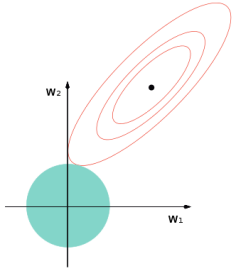
L2 vs. L1 regularization



$L2$, weight decay:

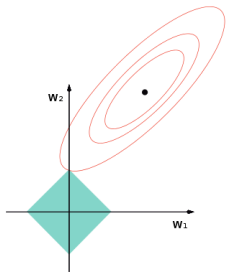
- "standard" regularizer
- differentiable (simple integration into gradient descent procedure)
- robust to noise (results in a "distributed" model, redundancies in weights allowed)
- non-sparse

L2 vs. L1 regularization



$L2$, weight decay:

- "standard" regularizer
- differentiable (simple integration into gradient descent procedure)
- robust to noise (results in a "distributed" model, redundancies in weights allowed)
- non-sparse



$L1$, Lasso, "sparsify":

- sparse (good for interpretable models, e.g. medical applications)
- non-differentiable at $w_i = 0 \forall i$ (but workaround via subgradient/least angle regression)
- less redundancies between weights $\rightsquigarrow$ more prone to noise

Where are we?

Regularization

Validation

Nonlinear basis functions

Validation

Recall: performance measure, a criterion for *model selection*.

$$E^G_{[\underline{\mathbf{w}}]} := \langle\, e \,\rangle_{y_T, \underline{\mathbf{x}}; \underline{\mathbf{w}}} = \iint d\underline{\mathbf{x}}\, dy_T\; P(y_T, \underline{\mathbf{x}})\, e(y_T, y\,(\underline{\mathbf{x}}; \underline{\mathbf{w}}))$$

Validation

Recall: performance measure, a criterion for *model selection*.

$$E^G_{[\underline{\mathbf{w}}]} := \langle e \rangle_{y_T, \underline{\mathbf{x}}; \underline{\mathbf{w}}} = \iint d\underline{\mathbf{x}}\, dy_T\, P(y_T, \underline{\mathbf{x}})\, e(y_T, y(\underline{\mathbf{x}}; \underline{\mathbf{w}}))$$

Because $P(y_T, \underline{\mathbf{x}})$ is not known,
empirical average $E^T_{[\underline{\mathbf{w}}]}$ using the available training data:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)} \right) \right\}, \alpha = 1, \ldots, p.$$

batch training error: $$E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p} \sum_{\alpha=1}^{p} \underbrace{e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}} \right) \right)}_{e^{(\alpha)}}$$

or mini-batch and stochastic online approximations of $E^T_{[\underline{\mathbf{w}}]}$
not limited to scalar attributes y, y_T.

Q: *How well does E^T represent E^G?*

A discussion about model validation

1. Diagnostics for how well E^T represents E^G
2. Improving estimates for the generalization error E^G
3. Common methods for improving generalization error E^G, doing better on unseen data (e.g. regularization)

Training error alone is not enough

$\underline{\mathbf{w}}^* = \mathrm{argmin}_{\underline{\mathbf{w}}}\, E^T_{[\underline{\mathbf{w}}]}$ does not imply good generalization performance.

$$\text{small } E^T_{[\underline{\mathbf{w}}^*]} \quad \not\Rightarrow \quad \text{small } E^G_{[\underline{\mathbf{w}}^*]}$$

Expected:

$$E^T_{[\underline{\mathbf{w}}^*]} \quad < \quad E^G_{[\underline{\mathbf{w}}^*]}$$

Validation: Estimating generalization performance

Validation: evaluate the generalization of a model (and its training procedure) by estimating

$$\widehat{E}^{G}_{[\underline{\mathbf{w}}]} \approx E^{G}_{[\underline{\mathbf{w}}]}$$

Validation: Estimating generalization performance

Validation: evaluate the generalization of a model (and its training procedure) by estimating

$$\widehat{E}^G_{[\underline{\mathbf{w}}]} \approx E^G_{[\underline{\mathbf{w}}]}$$

This implies that we need a set of data.

- $\{\underline{\mathbf{x}}^{(\alpha)}_{\text{train}}, y_T{}^{(\alpha)}_{\text{train}}\}_{\alpha=1}^{p}$ for model selection
- $\rightsquigarrow \underline{\mathbf{w}}^* = \operatorname{argmin}_{\underline{\mathbf{w}}} E^T_{[\underline{\mathbf{w}}]}$

Validation: Estimating generalization performance

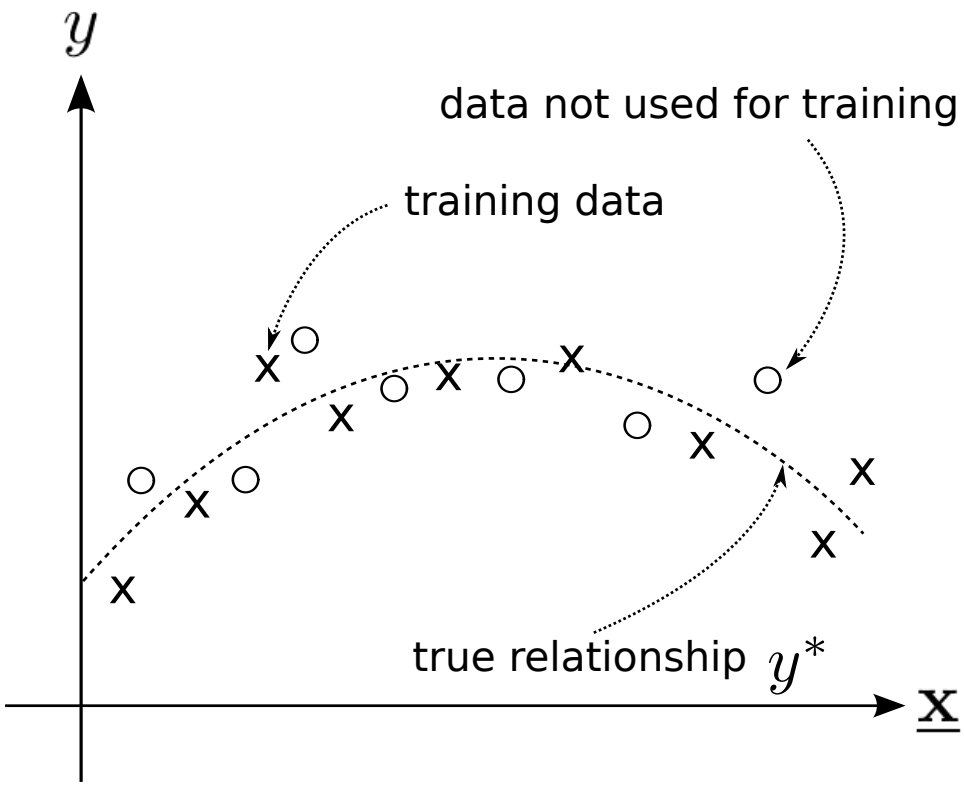
Validation: evaluate the generalization of a model (and its training procedure) by estimating

$$\widehat{E}^G_{[\underline{\mathbf{w}}]} \approx E^G_{[\underline{\mathbf{w}}]}$$

This implies that we need a set of data.

- $\{\underline{\mathbf{x}}^{(\alpha)}_{\text{train}}, y_T{}^{(\alpha)}_{\text{train}}\}^p_{\alpha=1}$ for model selection
- $\rightsquigarrow \underline{\mathbf{w}}^* = \operatorname{argmin}_{\underline{\mathbf{w}}} E^T_{[\underline{\mathbf{w}}]}$
- $\{\underline{\mathbf{x}}^{(\beta)}_{\text{val.}}, y_T{}^{(\beta)}_{\text{val.}}\}^q_{\beta=1}$ for validation
- $\rightsquigarrow \widehat{E}^G_{[\underline{\mathbf{w}}^*]}$

Overfitting vs. Underfitting

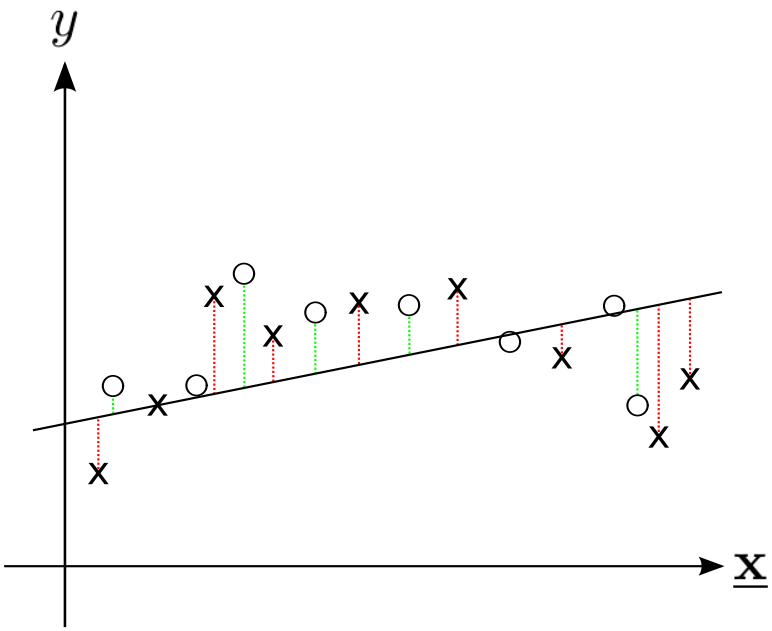


data generation:

e.g. $y_T = y^*_{(\underline{\mathbf{x}};\underline{\mathbf{w}})} + \underbrace{\eta}_{\text{noise, zero mean}}$

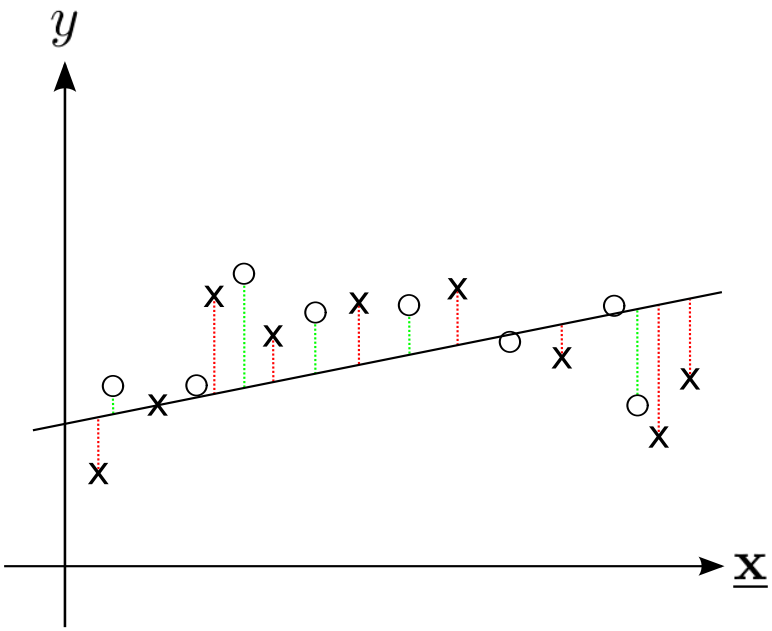
Overfitting vs. Underfitting

Compare E^T and $\widehat{E}^G$:



Overfitting vs. Underfitting

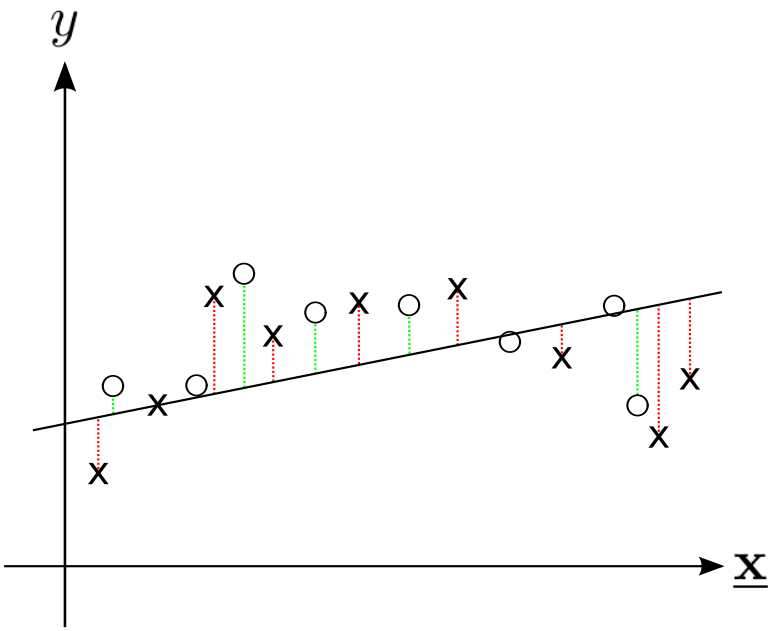
Compare E^T and $\widehat{E}^G$:

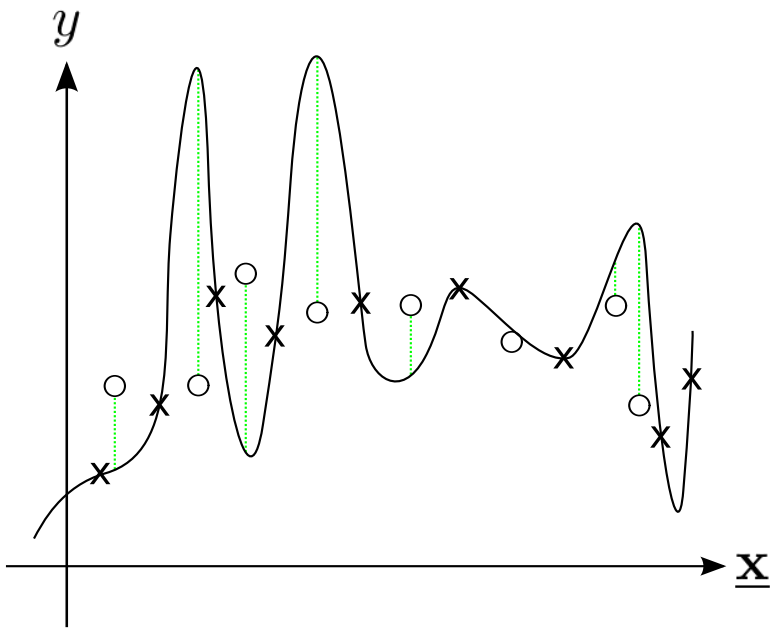


$$\left.\begin{array}{l} E^T \text{ large} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \approx \widehat{E}^G$$

Overfitting vs. Underfitting

Compare E^T and $\widehat{E}^G$:

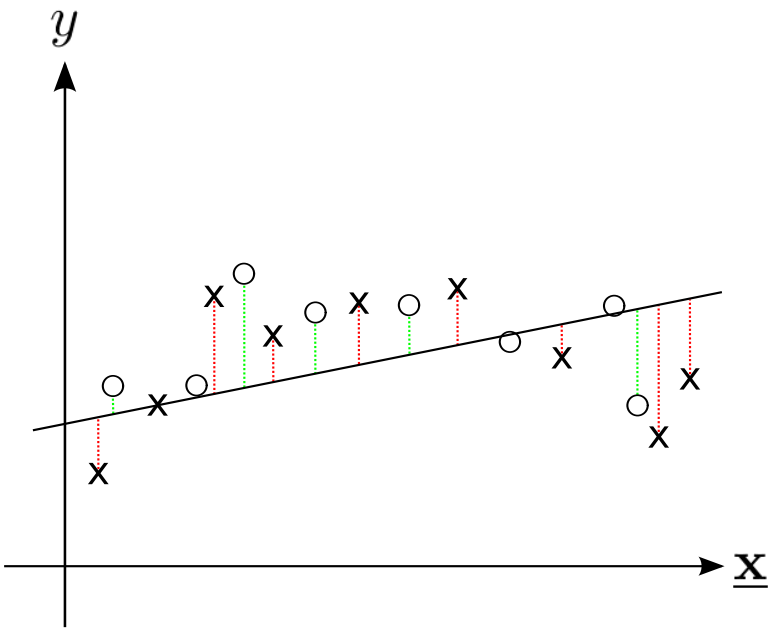


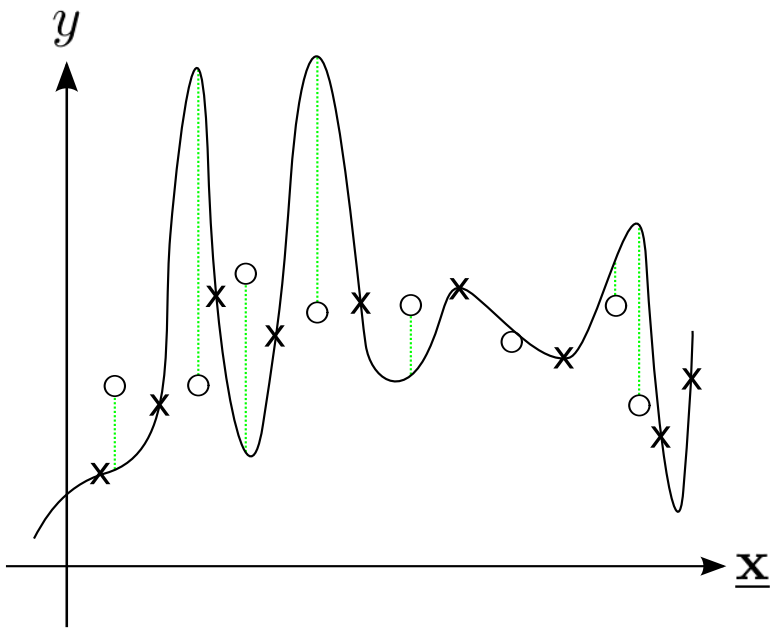


$$\left.\begin{array}{l} E^T \text{ large} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \approx \widehat{E}^G$$

Overfitting vs. Underfitting

Compare E^T and $\widehat{E}^G$:

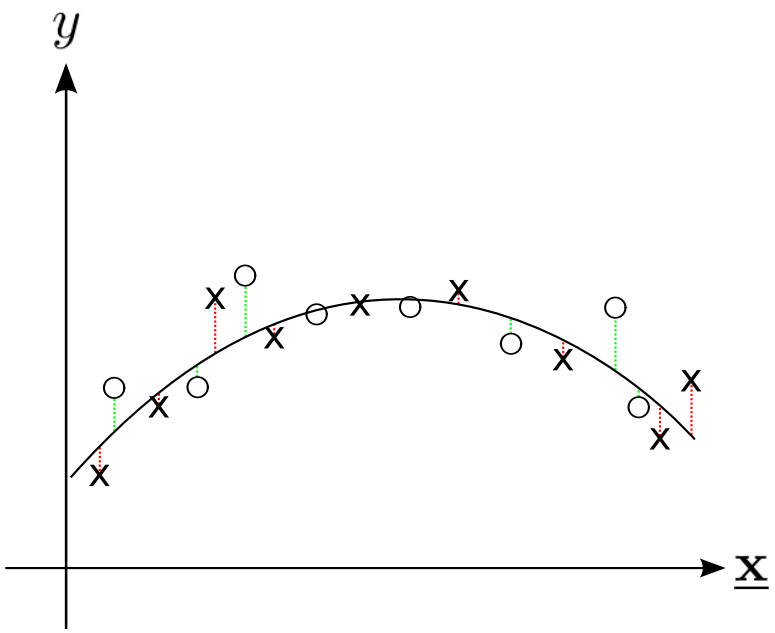




$$\left.\begin{array}{l} E^T \text{ large} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \approx \widehat{E}^G$$

$$\left.\begin{array}{l} E^T \text{ small} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \ll \widehat{E}^G$$

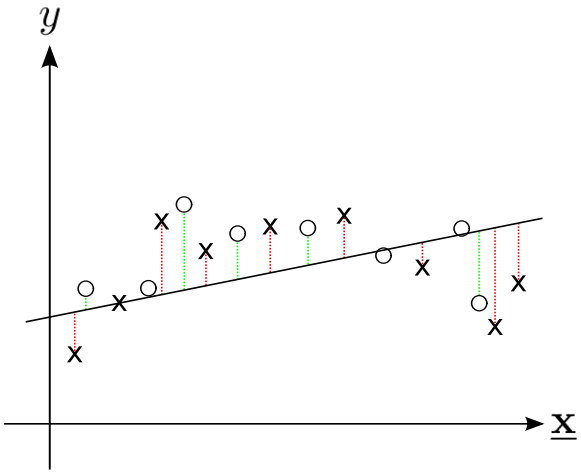
Well fitted model



Diagnostics

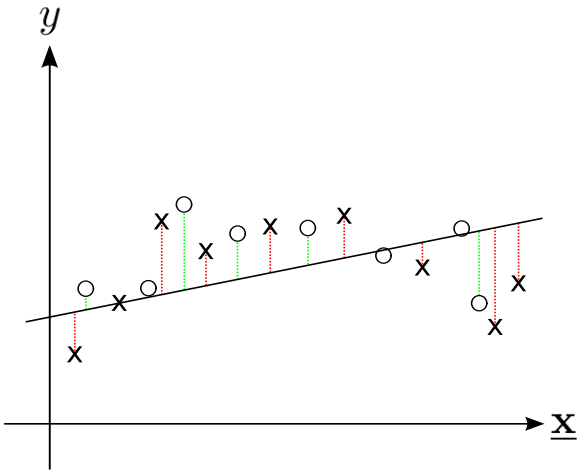
$$\left.\begin{array}{l} E^T \text{ small} \\ \widehat{E}^G \text{ small} \end{array}\right\} E^T \approx \widehat{E}^G$$

Underfitting



$$\left.\begin{array}{l} E^T \text{ large} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \approx \widehat{E}^G$$

Underfitting

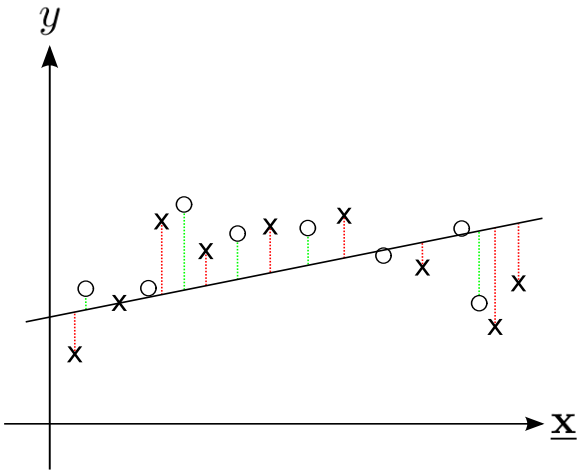


$$\left.\begin{array}{l} E^T \text{ large} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \approx \widehat{E}^G$$

Q: *What is underfitting?*

- function complexity vs. model complexity
- the degrees of freedom in the model are too ...

Underfitting



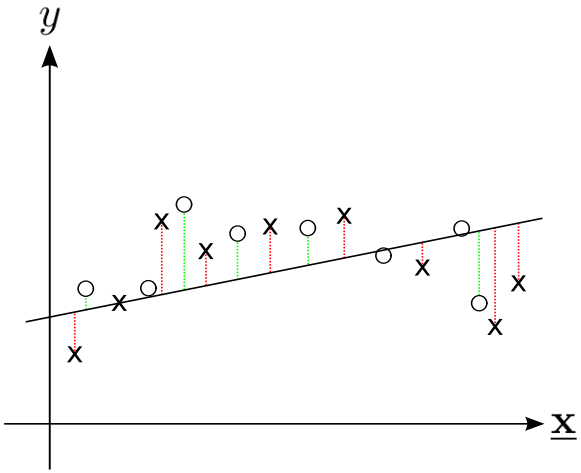
$$\left.\begin{array}{l} E^T \text{ large} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \approx \widehat{E}^G$$

Q: *What is underfitting?*

- function complexity vs. model complexity
- the degrees of freedom in the model are too ...
- the model is insensitive to variations in the features

Q: *How would replacing a few training samples change the solution?*

Underfitting



$$\left.\begin{array}{l} E^T \text{ large} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \approx \widehat{E}^G$$

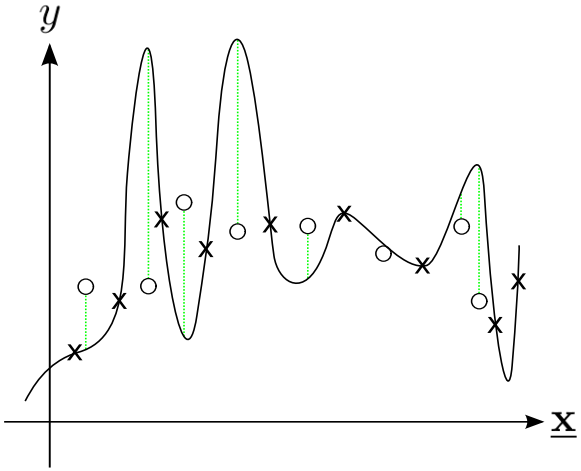
Q: *What is underfitting?*

- function complexity vs. model complexity
- the degrees of freedom in the model are too ...
- the model is insensitive to variations in the features

Q: *How would replacing a few training samples change the solution?*

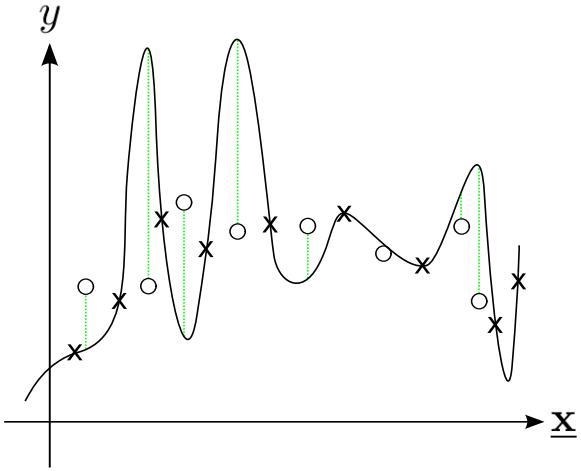
- Not much.

Overfitting



$$\left.\begin{array}{l} E^T \text{ small} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \ll \widehat{E}^G$$

Overfitting

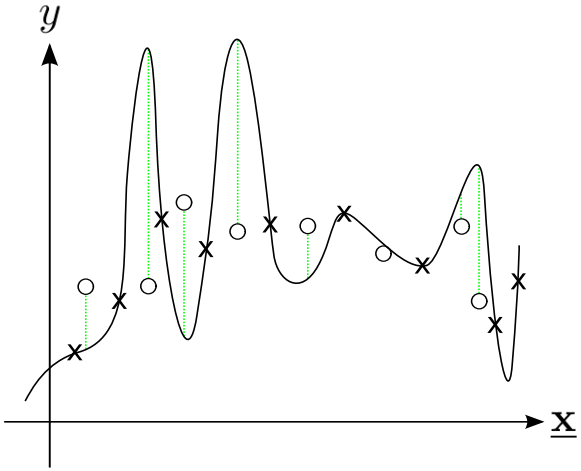


$$\left.\begin{array}{l} E^T \text{ small} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \ll \widehat{E}^G$$

Q: *What is overfitting?*

- function complexity vs. model complexity

Overfitting



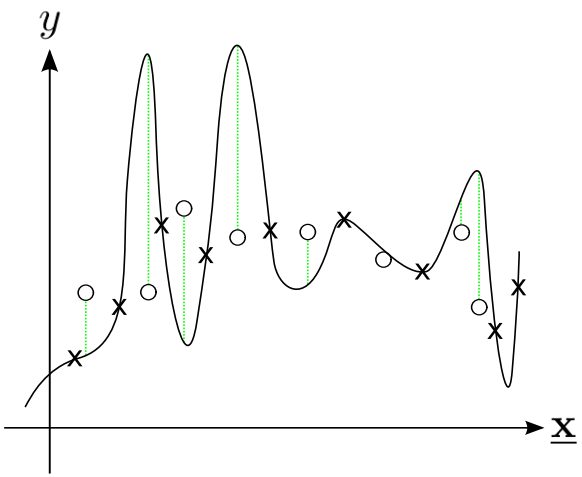
$$\left.\begin{array}{l} E^T \text{ small} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \ll \widehat{E}^G$$

Q: *What is overfitting?*

- function complexity vs. model complexity
- the degrees of freedom in the model are...

Overfitting

Q: *What is overfitting?*



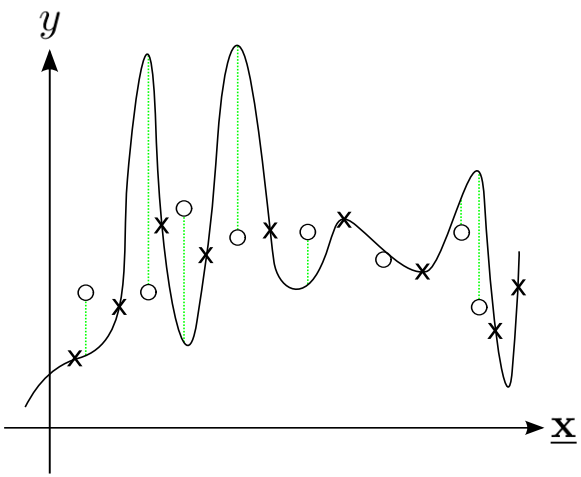
$$\left.\begin{array}{l} E^T \text{ small} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \ll \widehat{E}^G$$

- function complexity vs. model complexity
- the degrees of freedom in the model are...
- the model is over-sensitive to the noise component
- the model has "memorized" the training data instead of learning a representation for it.

Q: *How would replacing a few training samples change the solution?*

Overfitting

Q: *What is overfitting?*



$$\left.\begin{array}{l} E^T \text{ small} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \ll \widehat{E}^G$$

- function complexity vs. model complexity
- the degrees of freedom in the model are...
- the model is over-sensitive to the noise component
- the model has "memorized" the training data instead of learning a representation for it.

Q: *How would replacing a few training samples change the solution?*

Dealing with overfitting/underfitting

	reduces $\rightarrow$	overfitting	underfitting
1	more training data		
2	larger reg. coeff. λ		
3	smaller reg. coeff. λ		
4	use a more complex model		
5	use a simpler model		
6	increase no. of hidden neurons in MLP		
7	add more hidden layers to MLP		
8	add features to the input $\underline{\mathbf{x}}$		
9	shuffle the data		

Dealing with overfitting/underfitting

	potentially reduces →	overfitting	underfitting
1	more training data	yes	no
2	larger regularization coeff. λ	yes	no
3	smaller regularization coeff. λ	no	yes
4	use more complex model	no	yes
5	use simpler model	yes	no
6	increase number of hidden neurons in MLP	no	yes
7	add more hidden layers to MLP	no	yes
8	add features to the input $\underline{\mathbf{x}}$	no	yes
9	shuffle the data	no	no
	more techniques?		

Different views of the bias-variance tradeoff

Remember: it's always a bias-variance <u>tradeoff</u>

Different views of the bias-variance tradeoff

Change the size of the training set p to identify potential overfitting vs. underfitting of the model:

Q: *What* E^T *vs.* p *relationship do we expect for an underfitting model?*

Different views of the bias-variance tradeoff

Change the size of the training set p to identify potential overfitting vs. underfitting of the model:

Q: *What* E^T *vs.* p *relationship do we expect for an underfitting model?*

Q: *What* E^T *vs.* p *relationship do we expect for an overfitting model?*

Different views of the bias-variance tradeoff

Change the size of the training set p to identify potential overfitting vs. underfitting of the model:

Q: *What* E^T *vs.* p *relationship do we expect for an underfitting model?*

Q: *What* E^T *vs.* p *relationship do we expect for an overfitting model?*

Q: *What* $\widehat{E}^G$ *vs.* p *relationship do we expect for an underfitting model?*

Different views of the bias-variance tradeoff

Change the size of the training set p to identify potential overfitting vs. underfitting of the model:

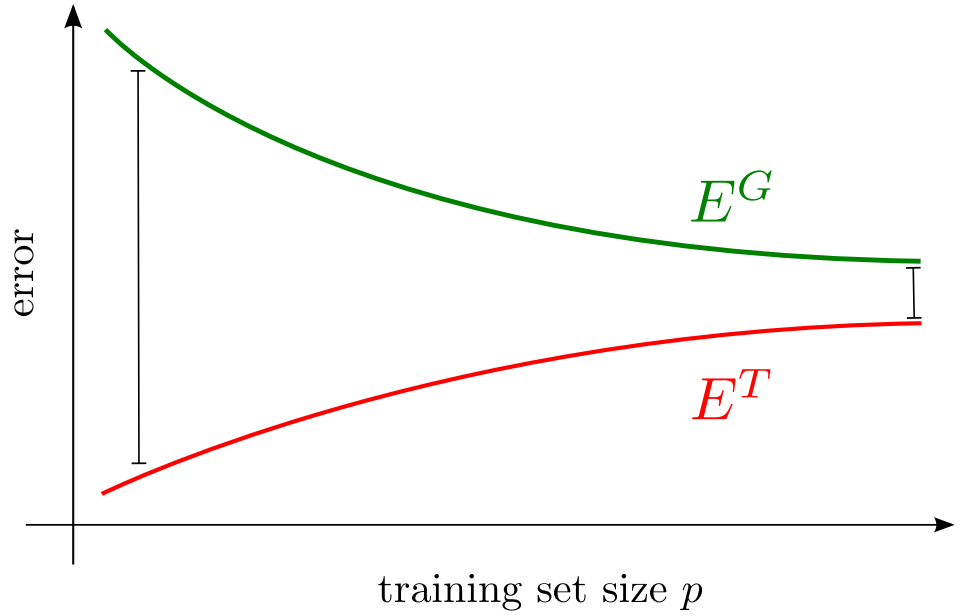
Q: *What* E^T *vs.* p *relationship do we expect for an underfitting model?*

Q: *What* E^T *vs.* p *relationship do we expect for an overfitting model?*

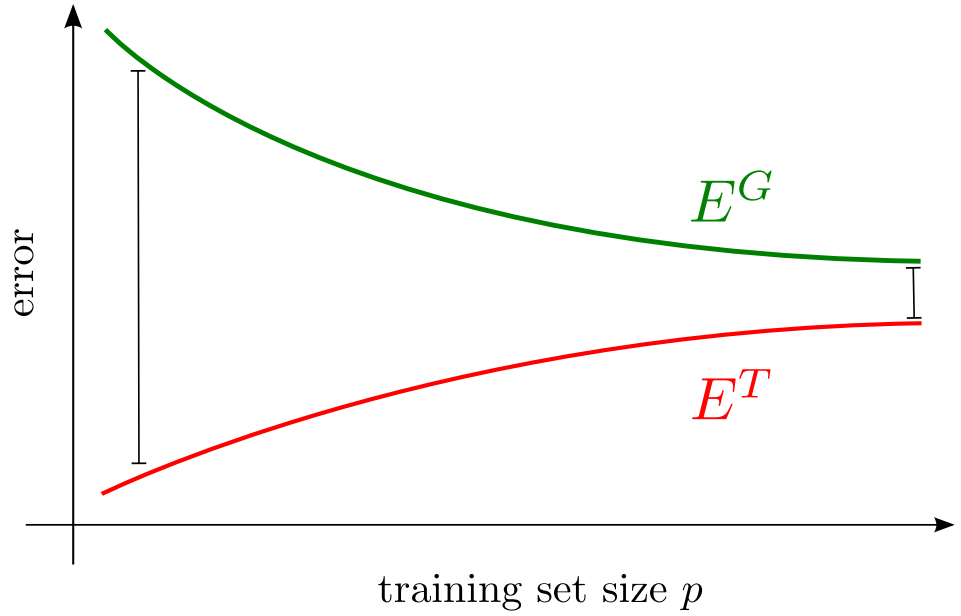
Q: *What* $\widehat{E}^G$ *vs.* p *relationship do we expect for an underfitting model?*

Q: *What* $\widehat{E}^G$ *vs.* p *relationship do we expect for an overfitting model?*

Different views of the bias-variance tradeoff

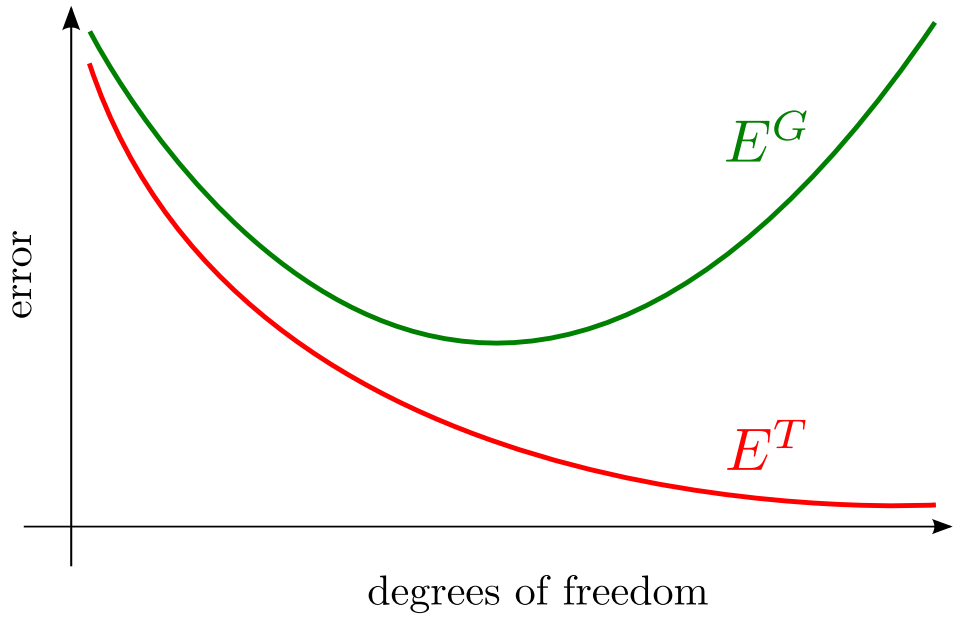


Different views of the bias-variance tradeoff

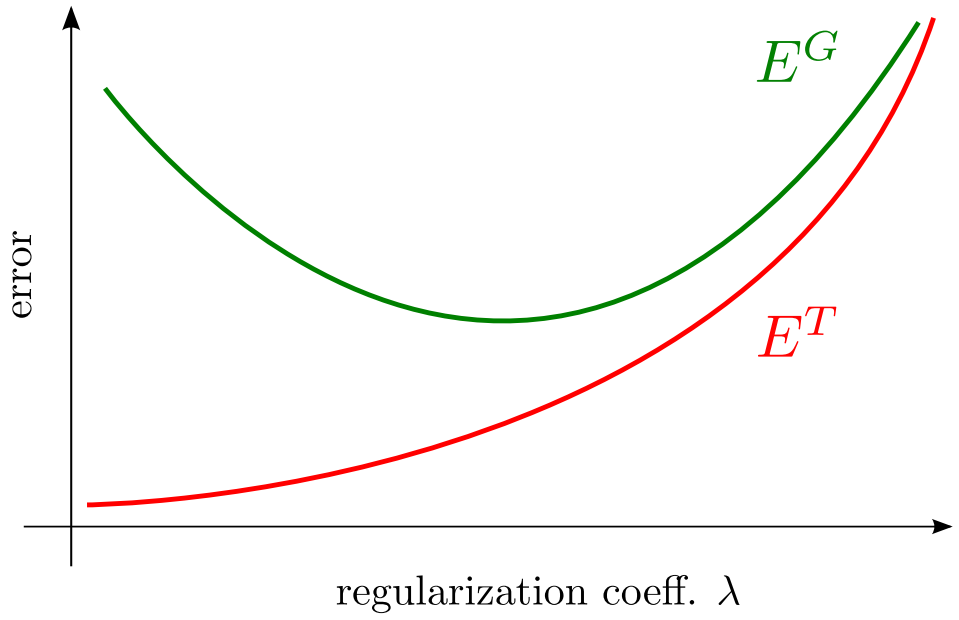


Q: *What is the benefit of looking at the bias-variance tradeoff as a function of p?*

Different views of the bias-variance tradeoff



Different views of the bias-variance tradeoff



Improving how we estimate $\widehat{E}^G$

Validation: Testset method

Testset method (with hyperparameters, e.g. regularization coeff. λ)

3-way split of the data.

1. training set: perform model selection for different values of λ
2. test set: select value of λ, which provides best prediction results on this set $\rightsquigarrow \lambda^{\text{opt}}$
3. validation set: estimate generalization performance of selected λ
   This is data that was not used for model selection or hyperparameter selection.

training set	test set	validation set

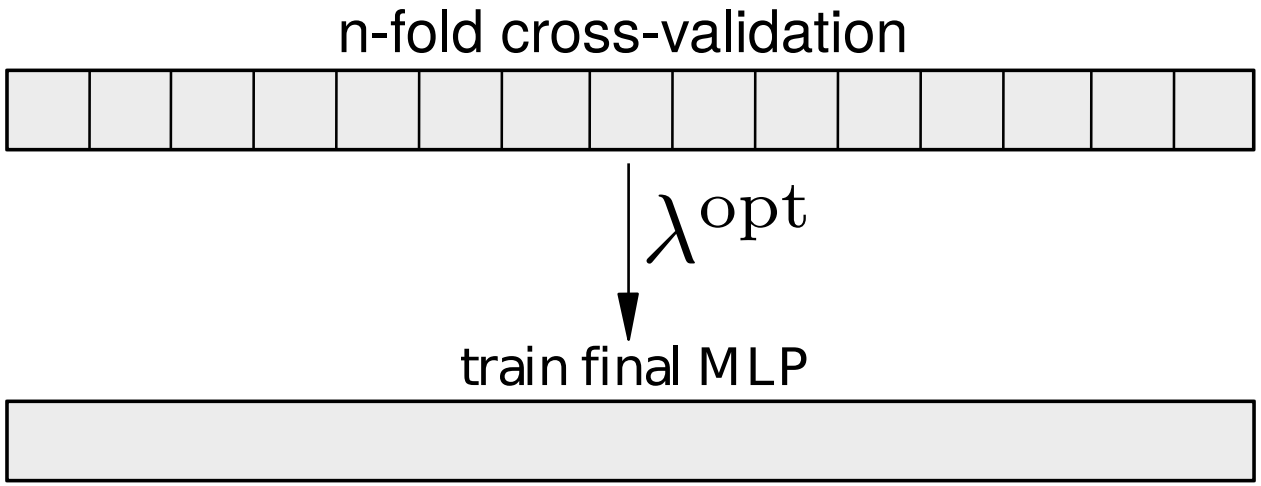
Potential overfitting of λ^{opt} to test set.

n-fold cross-validation for hyperparameter selection

for $\lambda = \lambda_1$ **to** λ_n **do**
  perform n-fold cross-validation on data with regularization λ
end

pick optimal λ^{opt} with minimum $\widehat{E}^G = \langle \widehat{E}^G_k \rangle_{n\text{folds}}$

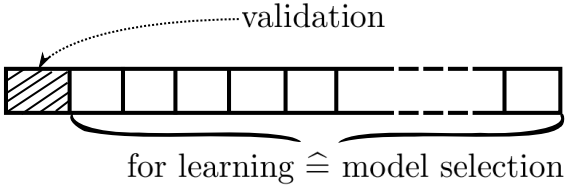
final model: train network on all data with λ^{opt}



Optional: Additional validation set for estimating $\widehat{E}^G$ of final model.

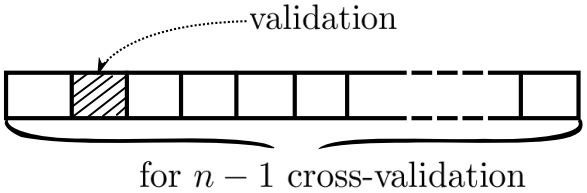
Validation: nested cross-validation

1



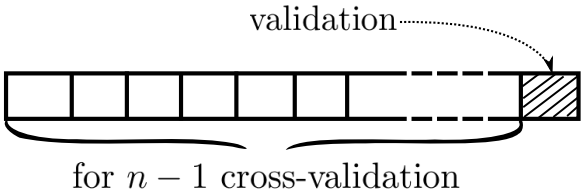
- do n - 1 cross-validation for all values of λ
- train with best λ
- validate with learned model

2



- do n - 1 cross-validation for all values of λ
- train with best λ
- validate with learned model

n



- do n - 1 cross-validation for all values of λ
- train with best λ
- validate with learned model

- $\widehat{E}^G \quad \widehat{=} \quad$ average over all n validation errors

- Never use test data for model selection.
- Always embed the whole selection procedure (including hyper-parameter search) within an n-fold cross-validation run.
- Watch out for validation data "leaking" into the training set.

Still with me?

Regularization

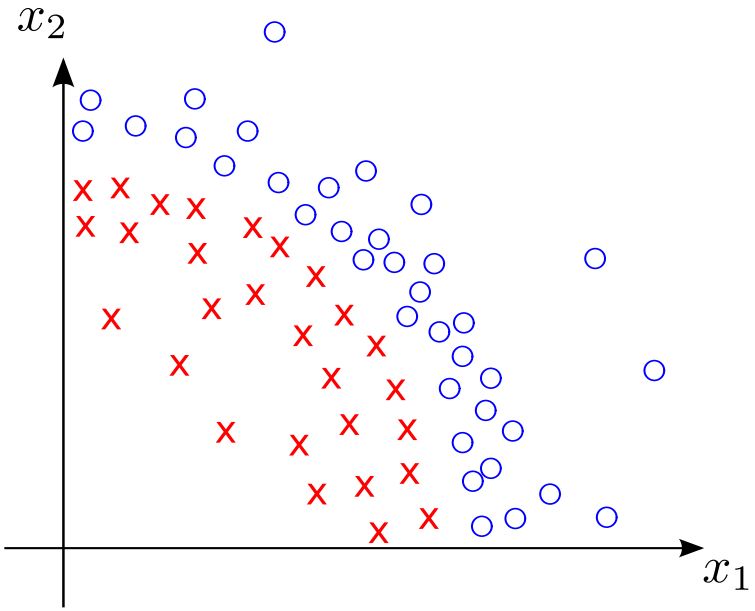
Validation

Nonlinear basis functions

- solve non-linear problems
- identify the bias-variance tradeoff
- regularize our model

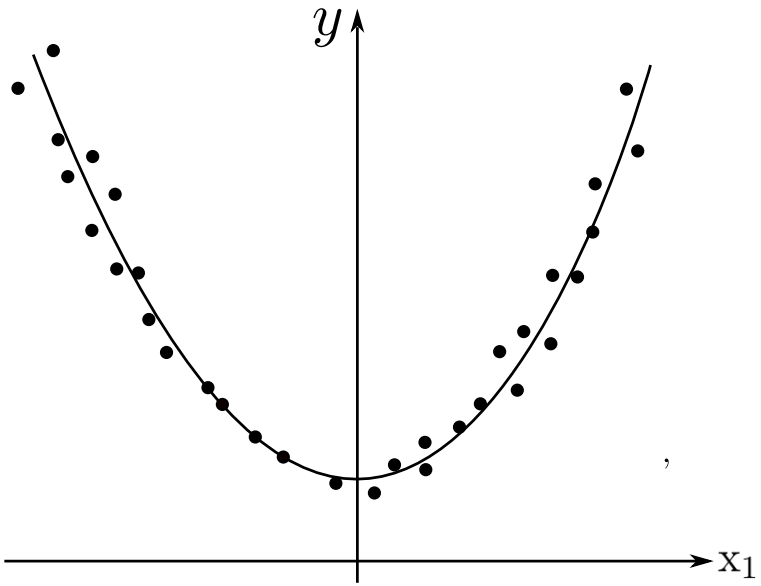
A non-linearly separable problem

Q: *Find a perfect linear separation between the classes using only a connectionist neuron.*



A non-linearly separable problem

Q: *Fit this parabola using only a connectionist neuron.*

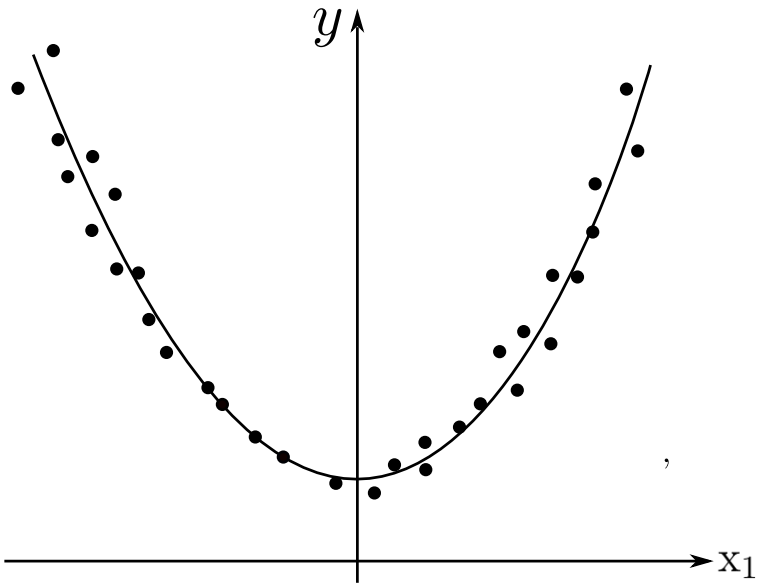


Q: *How would you characterize any parabolic (quadratic) function?*

$$y(x) =$$

A non-linearly separable problem

Q: *Fit this parabola using only a connectionist neuron.*



Q: *How would you characterize any parabolic (quadratic) function?*

$$y(x) = a \cdot x^2 + b \cdot x + c = w_2 \cdot x^2 + w_1 \cdot x + w_0$$

Nonlinear basis function

Feature transformation ($\underline{\mathbf{x}} \in \mathbb{R}^N$):

$$\underline{\phi} : \underline{\mathbf{x}} \mapsto \underline{\phi}(\underline{\mathbf{x}})$$

Expanding our feature space using *Monomials* of highest degree k (i.e. polynomial expansion):

$$\phi_i(\underline{\mathbf{x}}) \in \{\ 1, x_1\ ,\ x_2, \ldots, x_N, x_1^2\ ,\ x_1 x_2, \ldots, x_1 x_N, \ldots,\ x_N^2,\ \ldots \\ x_1 x_N^{k-1}\ ,\ x_1^k, \ldots, x_N^k\}$$

$$\phi_i(\underline{\mathbf{x}}) \in \left\{ \prod_{j=1}^{N} x_j^{a_j} \quad : a_j \in \mathbb{N}_0 \ \text{ with } \ 0 \leq \sum_{j=1}^{N} a_j \leq k \right\}$$

where $i = 1, \ldots d$.
Example: $k = 9, N = 2 \quad \Rightarrow \quad d = 55$

Preprocessing: Sphering

Large $\|\underline{\mathbf{x}}\|$ can lead to very large $\phi_i(\underline{\mathbf{x}})$.
We want to decorrelate the transformed data and standardize it (keep variance of $x_i = 1$) For every point $\alpha = 1, \ldots, p$ from the training set:

$$\underline{\mathbf{x}}^{(\alpha)}_{\text{sphered}} = \underline{\mathbf{\Lambda}}^{-\frac{1}{2}} \underline{\mathbf{E}}^{\top} \underline{\mathbf{x}}^{(\alpha)}_{\text{centered}}.$$

Here

$\underline{\mathbf{x}}^{(\alpha)}_{\text{centered}} = \underline{\mathbf{x}}^{(\alpha)} - \langle \underline{\mathbf{x}} \rangle \qquad \langle \underline{\mathbf{x}} \rangle = \frac{1}{p} \sum_{\alpha=1}^{p} \underline{\mathbf{x}}^{(\alpha)}$,
$\underline{\mathbf{E}} = (\underline{\mathbf{e}}_1, \ldots, \underline{\mathbf{e}}_N)$ is the eigenvector matrix and
$\underline{\mathbf{\Lambda}} = \operatorname{diag}(\lambda_1, \ldots, \lambda_N)$ is the eigenvalue matrix for the eigendecomposition

$$\underline{\mathbf{C}}\, \underline{\mathbf{e}}_i = \lambda_i \underline{\mathbf{e}}_i$$

of the covariance matrix $\underline{\mathbf{C}}$ with $C_{ij} = \frac{1}{p} \sum_{\alpha=1}^{p} x^{(\alpha)}_{\text{centered},i}\, x^{(\alpha)}_{\text{centered},j}$.

Preprocessing: Sphering

Examining the properties of the data after sphering:

$$\frac{1}{p}\sum_{\alpha=1}^{p} \underline{\mathbf{x}}^{(\alpha)}_{\text{sphered}} = \underline{\mathbf{0}} \quad \text{(centered)},$$

$$\frac{1}{p}\sum_{\alpha=1}^{p} (x^{(\alpha)}_{\text{sphered,i}})^2 = 1 \quad \text{(unit variance)},$$

$$\frac{1}{p}\sum_{\alpha=1}^{p} x^{(\alpha)}_{\text{sphered},i}\, x^{(\alpha)}_{\text{sphered},j} = 0 \quad \text{(zero covariance)},$$

Important: Reuse the same pre-processing (centering, sphering) on validation set. Also apply to test-set in case of a 3-way split.

Linear neuron for regression: Analytical solution

Data:

transformed input	$\underline{\phi}^{(\alpha)} := \underline{\phi}(\underline{\mathbf{x}}^{(\alpha)}) = \left(\phi_1(\underline{\mathbf{x}}^{(\alpha)}), \phi_2(\underline{\mathbf{x}}^{(\alpha)}), \ldots, \phi_d(\underline{\mathbf{x}}^{(\alpha)})\right)^\top$
label	$y_T^{(\alpha)} \in \mathbb{R}$
with	$\alpha = 1, \ldots, p$ and $\underline{\mathbf{x}}$ has been centered and sphered.

Linear neuron for regression: Analytical solution

Data:

transformed input	$\underline{\phi}^{(\alpha)} := \underline{\phi}(\underline{\mathbf{x}}^{(\alpha)}) = \left(\phi_1(\underline{\mathbf{x}}^{(\alpha)}), \phi_2(\underline{\mathbf{x}}^{(\alpha)}), \ldots, \phi_d(\underline{\mathbf{x}}^{(\alpha)})\right)^\top$
label	$y_T^{(\alpha)} \in \mathbb{R}$
with	$\alpha = 1, \ldots, p$ and $\underline{\mathbf{x}}$ has been centered and sphered.

Model:

$$y(\underline{\phi}; \underline{\mathbf{w}}) = \underline{\mathbf{w}}^\top \underline{\phi} = \sum_{i=1}^{d} w_i \phi_i(\underline{\mathbf{x}})$$

quadratic cost function, L_2 regularization.

Linear neuron for regression: Analytical solution

Data:

transformed input	$\underline{\phi}^{(\alpha)} := \underline{\phi}(\underline{\mathbf{x}}^{(\alpha)}) = \left(\phi_1(\underline{\mathbf{x}}^{(\alpha)}), \phi_2(\underline{\mathbf{x}}^{(\alpha)}), \ldots, \phi_d(\underline{\mathbf{x}}^{(\alpha)})\right)^\top$
label	$y_T^{(\alpha)} \in \mathbb{R}$
with	$\alpha = 1, \ldots, p$ and $\underline{\mathbf{x}}$ has been centered and sphered.

Model:

$$y(\underline{\phi}; \underline{\mathbf{w}}) = \underline{\mathbf{w}}^\top \underline{\phi} = \sum_{i=1}^{d} w_i \phi_i(\underline{\mathbf{x}})$$

quadratic cost function, L_2 regularization. Reuse solution for regularized linear regression:

$$\underline{\mathbf{w}}^* = \left(\underline{\mathbf{\Phi}}\,\underline{\mathbf{\Phi}}^\top + \lambda \underline{\mathbf{I}}_d\right)^{-1} \underline{\mathbf{\Phi}}\,\underline{\mathbf{y}}_{\text{True}}^\top$$

where $\underline{\mathbf{\Phi}} = \left(\underline{\phi}^{(1)}, \ldots, \underline{\phi}^{(\alpha)}, \ldots, \underline{\phi}^{(p)}\right) \in \mathbb{R}^{d,p}$